

19

Becoming a Wallet Other Apps Can Talk To

WHAT YOU WILL LEARN IN THIS CHAPTER

- ▶ The difference between being a *wallet* and being a *signing service*, and why every "real" wallet must be the second
- ▶ What Mobile Wallet Adapter (MWA) is, who built it, and why it exists separately from WalletConnect and EIP-1193
- ▶ How a dapp on the same phone discovers and talks to your wallet — the URL-scheme handshake, the session keypair, the ECDH-derived encryption key
- ▶ The four operations every MWA-compliant wallet must implement: `authorize`, `reauthorize`, `sign_transactions`, and `sign_and_send_transactions`
- ▶ Why MWA is half-Dart, half-native (Swift/Kotlin), and which half goes where
- ▶ How transaction-display works — the hardest UX problem in the wallet industry, in one section
- ▶ The threat model: phishing dapps, malicious payloads, replay attacks, and what your approval UI must defend against
- ▶ Why Solfare does not implement MWA today, and what would change for a v1 ship

Until this chapter, Solfare has been a self-contained app. The user opens it, sees their balance, sends SOL, swaps tokens. Every transaction in the book so far has been initiated by Solfare itself — the user tapped Send, Solfare built the transaction, Solfare signed it, Solfare submitted it.

That is not what real wallets do most of the time. Most of the time, a real wallet is a *signing service*. A user is browsing a marketplace called Magic Eden inside another app — maybe Safari, maybe a native dapp — and taps **Buy**. The marketplace builds a transaction. It hands that transaction to the user's wallet. The wallet shows the user what they are about to sign. The user approves. The wallet signs and returns the signature. The marketplace submits the transaction to the chain. The wallet never built the transaction. It only authorized it.

This is the model every wallet that matters supports. Phantom does it. Solflare does it. Backpack does it. Hardware wallets do it. The protocol that lets dapps on a phone talk to wallets on the same phone is called **Mobile Wallet Adapter**, or MWA, and it is the subject of this chapter.

Solfare, the app this book documents, does *not* implement MWA. It is a clone built to learn web3, and MWA is a multi-week project with as much native Swift and Kotlin as Dart. But understanding how MWA works is essential to understanding what a wallet actually *is*, and to making informed decisions about whether to ship it. This chapter walks through the protocol end to end, sketches the implementation Solfare would adopt if it went there, and is honest about the parts that are genuinely hard.

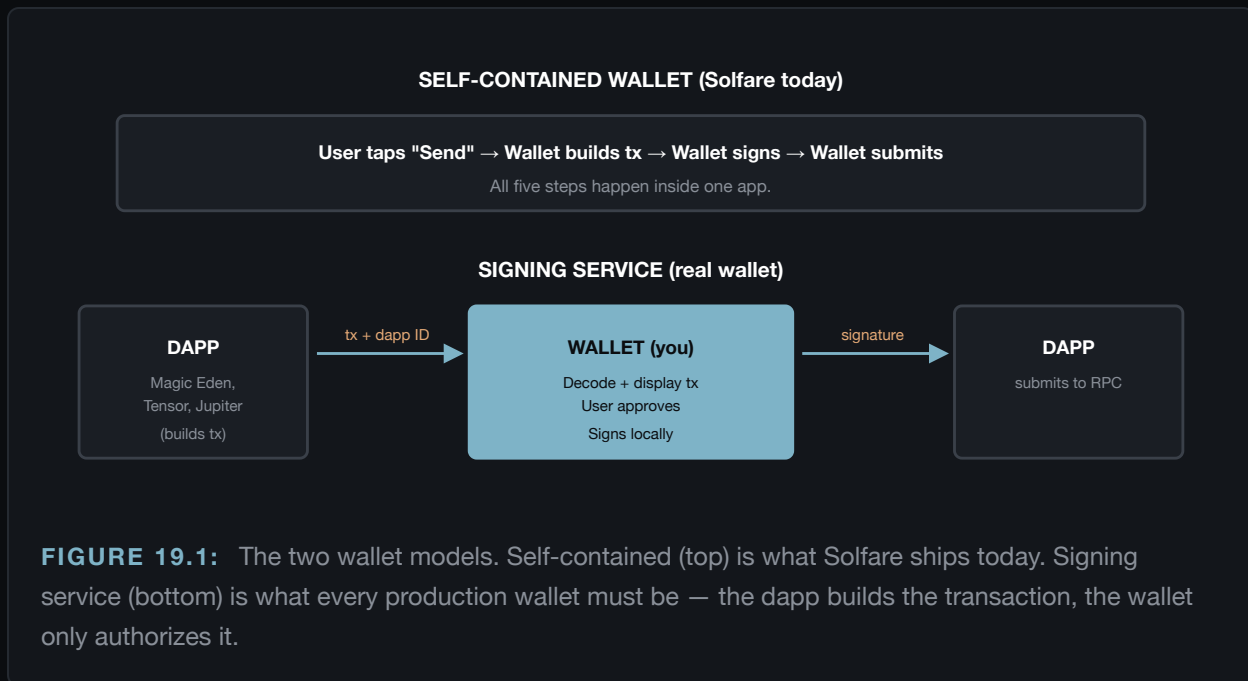
Wallet Versus Signing Service

Before MWA can make sense, you need to internalize a distinction the rest of this book has been quietly avoiding. A wallet, in the popular imagination, is "the thing that holds my crypto." That is wrong in a useful, productive way. A wallet does not hold anything. The blockchain holds your funds; your wallet holds the *private key* that proves you control them.

Think of it like this. The Solana network is a public ledger. An entry on that ledger says: *address ABC has 5 SOL*. Anyone can see that entry. To *move* that 5 SOL — to author a new ledger entry that says "address ABC sent 1 SOL to address XYZ" — you need a signature from address ABC's private key. That signature is the only thing the network cares about. It does not care which app produced it, what UI the user saw, or whether the user even understood what they signed. It cares that the signature is mathematically valid against the address's public key.

So a wallet is a thing that holds a private key and uses it to produce signatures. Everything else — balances, transaction lists, NFT galleries, swap interfaces — is decoration. The signature is the wallet's job.

A *self-contained wallet* like the Solfare we have built so far does both jobs in one app. It builds the transaction (decoration: send SOL, stake, swap) and then signs it. A *signing service*, which is what real wallets fundamentally are, can sign transactions someone else built. That someone else is usually a dapp.



Once you see the distinction, you also see why it is a security problem. The wallet, as signing service, is asked to sign payloads it did not construct, by software it did not write, on behalf of a user who probably cannot read the bytes. The wallet's job is not just to produce signatures — it is to *refuse* to produce them when the request looks dangerous, and to *show* the user what is happening when it is unclear. We will come back to this. It is the hardest part of the chapter and arguably the hardest part of running a wallet.

Why MWA, Not WalletConnect

If you come from Ethereum, you know WalletConnect — the protocol where you scan a QR code with your wallet, a relay server passes encrypted JSON-RPC calls between dapp and wallet, and signatures fly back across the relay. WalletConnect works on Solana too; several wallets support it. So why is there a separate Solana standard called Mobile Wallet Adapter?

Two reasons. First, WalletConnect was built for the case where the dapp and the wallet are on *different* devices — desktop dapp, mobile wallet — and need an internet relay to bridge them. When both are on the same phone, the relay is unnecessary overhead, an extra trust dependency, and a latency tax. MWA cuts the relay out: the two apps on the same phone talk directly via OS-level inter-app communication.

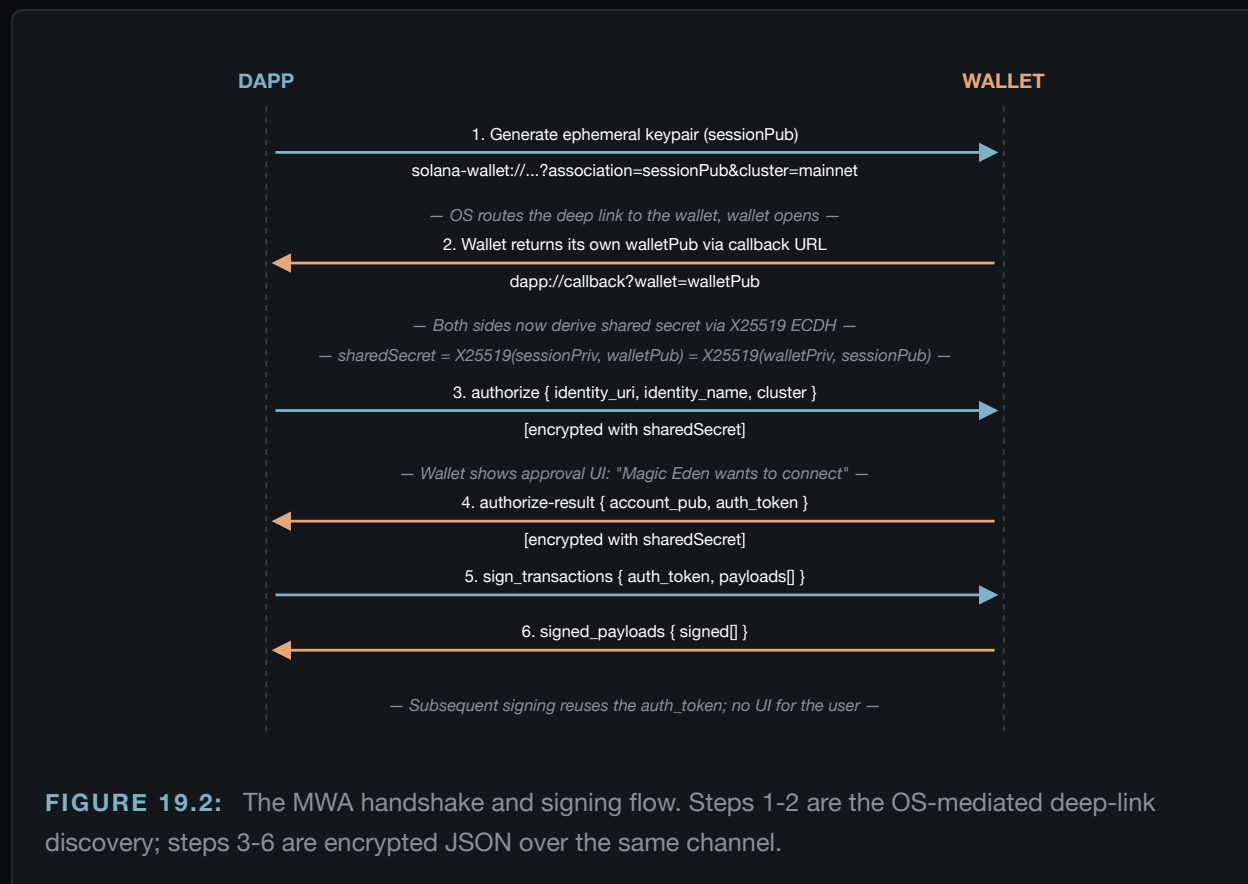
Second, Solana wanted an open standard that worked the same way on iOS and Android, that a wallet could implement once and have every dapp speak to. The Solana Mobile team — the people behind the Saga phone — published the MWA spec in 2022 and have iterated it since. It is genuinely open: the spec, the reference implementations, and the integration tests all live on GitHub under the `solana-mobile` organization.

WalletConnect and MWA are not mutually exclusive. A serious wallet implements both. WalletConnect handles the cross-device case. MWA handles the same-device case. Most user sessions today are same-device — a user opens a dapp in their browser on their phone and wants to sign something — so MWA is the more important of the two for mobile-first wallets.

The Handshake, Step by Step

MWA's most clever piece is the discovery handshake. There is no central registry of wallets. A dapp does not need to know in advance which wallets the user has installed. The protocol uses the operating system's deep-link machinery to do the discovery and the connection in one move.

Here is the flow, with both Android and iOS versions.



Step 1: Discovery via Deep Link

The dapp generates a fresh X25519 keypair — the *session keypair*. This keypair is one-time, used only for this connection. The dapp encodes the public half (32 bytes) into a URL that uses the `solana-wallet:` scheme, then asks the operating system to open that URL.

```
// Dapp side (any language). The session keypair lives only in memory.
final sessionKeys = X25519.generateKeyPair();
final uri = Uri.parse(
  'solana-wallet://?association='
  '${base64UrlEncode(sessionKeys.publicKey)}'
  '&cluster=mainnet-beta'
);
await launchUrl(uri);
```

What the OS does next depends on the platform. On Android, all installed apps that registered an `intent-filter` for the `solana-wallet` scheme are presented in a system disambiguation dialog. The user picks one. That app opens. On iOS, the OS uses *universal links* if the wallet has registered them, or falls back to a custom URL scheme that opens the most-recently-installed wallet (no picker — a real iOS limitation that is currently being addressed in the spec).

Either way, the wallet receives the `association` query parameter, decodes the dapp's session public key, and now has half of what it needs to derive a shared secret.

Step 2: Wallet Returns Its Public Key

The wallet generates its own ephemeral X25519 keypair, then opens the dapp's callback URL with the wallet's public key as a query parameter. On Android this works because the dapp provided a callback intent in the original request; on iOS it works because the dapp registered its own custom scheme that the wallet can target.

```
// Wallet side. We do this immediately after receiving the deep link.
final walletKeys = X25519.generateKeyPair();
final callback = Uri.parse(
  '${dappCallbackUri}?wallet=${base64UrlEncode(walletKeys.publicKey)}'
);
await launchUrl(callback);
```

Both apps now have the other's public key. Both compute the same shared secret using X25519 ECDH:

```
final sharedSecret = X25519.computeSecret(
    myPrivateKey, theirPublicKey,
);
// Both sides now hold the same 32 bytes. This is the symmetric key
// that encrypts every subsequent message.
```

Why ECDH, not just plain HTTPS? Because the messages flow through the operating system's deep-link plumbing, not through a network. Anything that can read those URL invocations — a malicious app monitoring intents, a sufficiently privileged debug tool — would otherwise see the request payloads. Encrypting end-to-end means even the OS cannot read the contents of the signing request.

Step 3-6: The Actual Conversation

From here on, every message between dapp and wallet is a JSON object encrypted with the shared secret. The protocol is JSON-RPC 2.0 (just like the Solana network's RPC, which we covered in Chapter 4). Table 19.1 lists the methods every MWA-compliant wallet must implement.

TABLE 19.1: The four required MWA methods.

METHOD	WHAT IT DOES
<code>authorize</code>	The dapp's first call. Identifies itself (URI, display name, icon URL) and asks the wallet to grant access to one account. The wallet shows a connection-approval screen. On approval, returns the user's chosen public key plus an <code>auth_token</code> the dapp will use for subsequent calls (so the user is not asked to reapprove every signature).
<code>reauthorize</code>	Refresh an expired or revoked <code>auth_token</code> . Same UX as <code>authorize</code> but with the dapp identity preserved — the user usually sees a lighter "still want to be connected?" prompt.
<code>sign_transactions</code>	Sign one or more transaction payloads without submitting them. Returns signed payloads; the dapp submits them itself. Used by dapps that want fine-grained control over submission timing.
<code>sign_and_send_transactions</code>	Sign and submit in one step. Returns transaction signatures (the txid). Most dapps use this — fewer round trips, less for the dapp to manage.

There are optional methods too — `sign_messages` for signing arbitrary bytes (used for "log in with wallet" auth flows), `deauthorize` for the user to disconnect from the wallet side, `clone_authorization` for transferring sessions across wallet reinstalls. A v1 wallet can ship with just the four required methods and add the rest later.

Where Dart Ends and Native Begins

Here is the inconvenient truth: most of MWA cannot live in Dart. The deep-link registration, the intent-filter on Android, the universal-link entitlements on iOS — all of these are platform-level configuration, not code. The pieces that *are* code split roughly as follows.

TABLE 19.2: Where each MWA component lives in a Flutter wallet.

COMPONENT	PLATFORM	WHY THERE
URL scheme registration	iOS Info.plist, Android Manifest	OS-level configuration. No code involved.
Deep-link receiver	AppDelegate.swift / MainActivity.kt → MethodChannel → Dart	Native handlers receive the URL, then forward into Dart. The plugin <code>app_links</code> abstracts this.
X25519 ECDH	Dart (<code>cryptography</code> package)	Pure Dart works here. No reason to drop down.
JSON-RPC encode/decode	Dart	Same as Chapter 4's RPC client — just a different transport.
Approval UI	Dart (Flutter)	This is just another Flutter screen. Solfare's existing modal sheets are 80% of what you need.
Transaction decoder	Dart (mostly)	Decoding the System Program, Token Program, and a few other common programs into human language is doable in Dart. Decoding arbitrary programs is impossible without an IDL — see §19.6.
Returning the callback URL	Dart, but executes via <code>url_launcher</code> which goes native	Dart calls <code>launchUrl</code> ; the OS handles the rest.

So: the heavy lifting is in Dart. The native pieces are configuration plus a thin receiver. The reason this still feels like a multi-week project is that you cannot test any of it without two apps — your wallet and a dapp — running simultaneously on the same phone, both correctly configured. Setting up that test harness is roughly half the work.

NOTE**Why "Solana Mobile Stack" Comes Up**

The Solana Mobile Stack (SMS) is the umbrella term for everything the Solana Mobile team built around the Saga phone: MWA, the Seed Vault (a hardware-backed key store), and the dApp Store (a side-loadable distribution channel). MWA is the only piece a wallet running on a normal phone needs. Seed Vault is Saga-specific. The dApp Store is for distribution, not runtime.

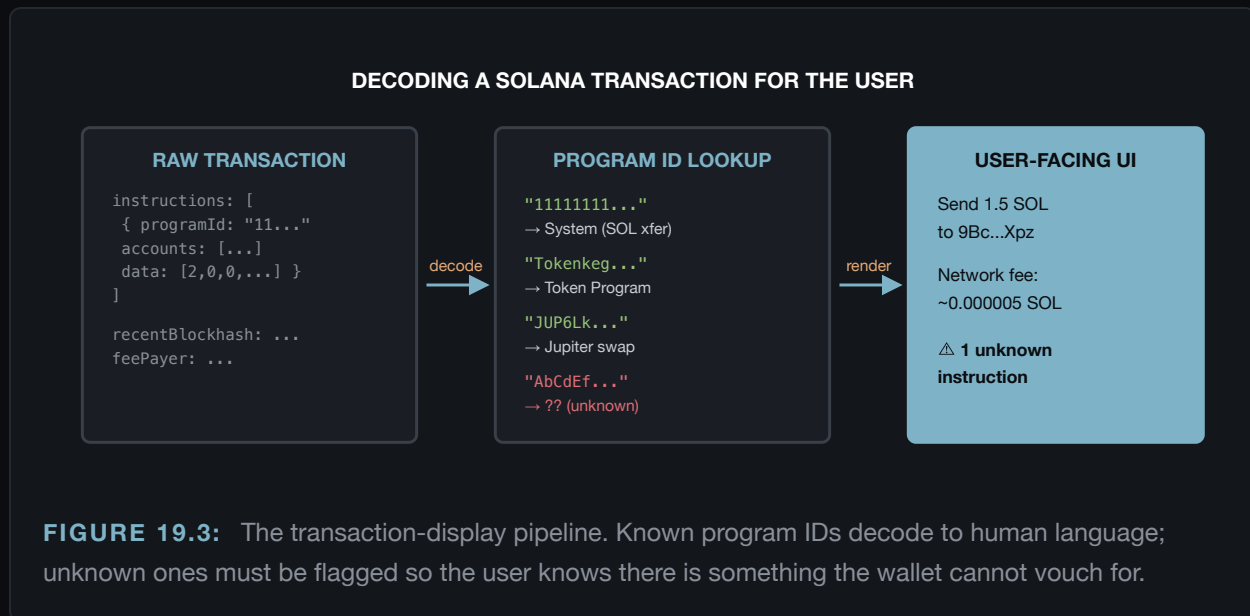
If you are reading the official MWA reference docs, you will see them assume Android. That is because Saga is Android-only, and the spec was written with Android intent semantics first. iOS support is a parallel implementation that achieves the same protocol over a different OS-level transport. Both speak the same JSON over the same encryption.

The Approval UI: The Hardest Problem in the Industry

An MWA wallet's user-facing surface is, when you strip away the chrome, exactly two screens. The *connection approval* screen, which the user sees once per dapp. And the *signing approval* screen, which the user sees every time the dapp asks for a signature. The first is easy. The second is, without exaggeration, the hardest UX problem the wallet industry has.

Here is the problem in concrete terms. The dapp hands you a transaction. It is a binary blob. You can decode the structure — instructions, accounts, program IDs, data. You can identify common programs: `11111111111111111111111111111111` is the System Program (transfer SOL), `TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA` is the Token Program (transfer SPL tokens), a few well-known program IDs map to known protocols (Jupiter for swaps, Marinade for staking).

For those known programs, you can decode the transaction into human language: "Send 1.5 SOL to address 9Bc...Xpz", or "Approve Token Program to transfer 10 USDC from your account." For everything else — and there are tens of thousands of programs deployed on Solana — you can show the program ID, the accounts it touches, and the raw data it carries, and let the user squint at it. Most users cannot read this. Most users approve anyway.



This is the wallet's hardest moment. If you display "Approve" with no warning when the transaction contains an unknown instruction, you have trained the user to approve everything, and a malicious dapp can drain their wallet by hiding a `SystemProgram.transfer` to the attacker's address inside an otherwise-innocent-looking swap. If you refuse to display anything you do not know how to decode, the user cannot use 90% of the dapp ecosystem.

Real wallets compromise. They render the parts they can. They show the parts they cannot as raw data, with a clear visual flag. They show simulation results from a *simulation RPC call* — a Solana RPC method that runs the transaction without committing it, returning the balance changes it would produce — so the user sees "after this transaction, your SOL balance will go from 5.0 to 3.5" even when the wallet cannot decode *why*. This simulation step is, in the author's opinion, the most important UX feature in any modern wallet.

WAR STORY

The Drainer Problem

In 2022 and 2023, several Solana wallets shipped MWA support but did not display unknown instructions clearly. A class of malicious sites called *drainers* exploited this: they looked like normal NFT mint pages, but the transaction they asked the user to sign actually contained a `SystemProgram.transfer` sending most of the user's SOL to an attacker. The wallet decoded the visible part (a token transfer for the supposed mint) and showed it. The user approved. The drain transfer ran first because it was instruction zero.

The fix was twofold: simulate every transaction before display, and surface the *net balance change* regardless of what instructions caused it. If the simulation says you will be 90% poorer after this signature, no decoded instruction list matters. The user needs to see the loss. Phantom and Solflare both ship this now. If you build an MWA wallet, it is non-negotiable.

What Solfare Would Add: A Sketch

If Solfare were going to ship MWA tomorrow, here is what would change. This is not implementation — Solfare deliberately stops short of MWA — but it is what an honest project plan would look like.

1. Native Configuration

iOS gets two new entries in `Info.plist`:

```
<key>CFBundleURLTypes</key>
<array>
  <dict>
    <key>CFBundleURLName</key>
    <string>com.solfare.mwa</string>
    <key>CFBundleURLSchemes</key>
    <array>
      <string>solana-wallet</string>
    </array>
  </dict>
</array>
```

Android gets an intent filter on `MainActivity`:

```

<activity android:name=".MainActivity" ... >
  <intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="solana-wallet" />
  </intent-filter>
</activity>

```

2. A New Bloc and Service

Solfare gets a new feature module: `lib/features/mwa/`. Inside, a `MwaService` handles the deep-link parsing, ECDH, and message encrypt/decrypt. A `MwaBloc` orchestrates the user-facing approval flow. The existing `WalletBloc` is untouched: `MwaBloc` calls into it via a method to fetch the active wallet's keypair when a signature is needed.

The handoff looks like this. `app_links` (a Flutter package that wraps native deep-link receivers on both platforms) fires a stream of incoming URLs. `MwaService` filters for `solana-wallet://` URLs, parses the association key, derives the shared secret, and emits a `MwaConnectionRequested` event. `MwaBloc` handles that event by emitting a `MwaApprovalNeeded` state, which the UI listens for and presents as a modal sheet.

3. The Approval Sheets

Two new screens. The *connection approval sheet* shows the dapp's name, icon, and identity URI, and asks the user to choose which of their wallets to authorize. (Solfare is multi-wallet; this sheet has a wallet picker.) The *signing approval sheet* shows the decoded transaction, the simulated balance change, and an Approve / Reject button pair. Both are `showModalBottomSheet` calls — the same primitive Solfare's existing send-confirm sheet uses.

4. The Decoder

A new file, `lib/features/mwa/data/transaction_decoder.dart`, that takes a base64-encoded transaction and returns a list of human-readable descriptions. It handles the System Program, Token Program, and Associated Token Account program natively. For unknown programs, it returns "Unknown instruction (program XXX)" and the UI flags it red. The simulation result comes from the existing `SolanaRpcDataSource`, with a new method `simulateTransaction` wrapping the Solana RPC `simulateTransaction` call.

The Threat Model You Are Now Inside

An MWA wallet's threat surface is much wider than a self-contained wallet's. Here are the four classes of attack you are now defending against, and what your wallet must do.

Phishing dapps impersonate legitimate ones. The user opened "Magic Eden" but it was actually `magiceden-airdrop.io`. Defense: show the dapp's identity URI prominently in both the connection sheet and the signing sheet. Do not use just the dapp-supplied name — show the URI too, large enough to read.

Malicious payloads are signed by legitimate dapps that have been compromised, or by fully malicious dapps. Defense: simulate every transaction, display the net balance change, flag unknown instructions. Never approve based on the dapp's reputation alone — even a trusted dapp can be hacked.

Replay attacks reuse an old `auth_token` after it should have expired. Defense: scope auth tokens to a single wallet account, expire them on a timer (typical: 24 hours), revoke them when the user changes the active wallet. Never let an auth token cross wallets.

Inter-app eavesdropping reads the deep-link URLs as they cross the OS. Defense: the ECDH-derived encryption key. This is why every message after the handshake is encrypted, even though the transport is "just" deep links on the same device.

Why Solfare Stops Here

You have read the architecture, the protocol, and the implementation sketch. You have seen Tables 19.1 and 19.2. You can now estimate what shipping MWA in Solfare would cost: roughly a thousand lines of new Dart, a few dozen lines each of Swift and Kotlin configuration, two new screens, one new bloc, one new service, and a transaction decoder of indefinite scope.

That is too much for a learning project. Solfare's purpose is to teach the wallet domain end-to-end; MWA would consume the entire remaining scope and still ship as a v0.1 because the transaction-decoder problem alone is multi-engineer-quarters of work to do well. So Solfare stops at the self-contained wallet. The reader, having read this chapter, can decide for themselves whether to extend.

If you do extend, start with deep-link reception and the approval sheet skeleton. Get a fake "dapp" — even just a button in your own debugger — to launch a `solana-wallet://` URL and show your approval sheet. From there, the rest of the protocol is a matter of filling in the JSON-RPC

method dispatch and the encryption layer. Save the transaction decoder for last; it is the largest piece, and your initial sheet can simply display "An unknown transaction is being requested" while you build it out.

Exercises

Exercise 19.1 — Read the Spec

Open the official MWA specification at github.com/solana-mobile/mobile-wallet-adapter and read it end to end. Note every place where the spec's behavior differs from the simplified description in this chapter. Most differences will be in the optional methods (`sign_messages`, `clone_authorization`) and the cluster-switching semantics. Write a short list of what you would add to this chapter for completeness.

Exercise 19.2 — Build a Test Dapp

Build a tiny Flutter app — separate from Solfare — that, on a button tap, generates a session key-pair and launches a `solana-wallet://` deep link. Install it alongside any production wallet (Phantom, Solflare). Verify your app can trigger the wallet's approval UI. You will not be able to complete the handshake (your dapp does not handle the callback), but you will see the wallet open and recognize the request.

Exercise 19.3 — Decode a Transaction by Hand

Take a base64-encoded transaction from any Solana explorer (Solscan, Solana FM). Decode the byte structure manually using the on-chain transaction format. Identify the instructions, the accounts, the program IDs. For each instruction, look up the program ID and write what the instruction *does* in plain English. Now imagine doing this in 200 milliseconds for any transaction the user is about to sign. That is what a transaction decoder must do.

Exercise 19.4 — Design the Approval Sheet

Sketch — on paper or in Figma — the signing approval sheet for a transaction that contains: (a) one known instruction (a SOL transfer), (b) one unknown instruction (some random program ID), and (c) a simulation result showing the user will lose 0.5 SOL net. What does the sheet display? What does the Approve button look like? Is the unknown instruction flagged? Is there a "details" disclosure for advanced users? This is the design problem MWA forces you to solve.

Summary

A real wallet is a signing service, not a self-contained app. The protocol that lets dapps on the same phone talk to a wallet is Mobile Wallet Adapter — a Solana-Mobile-led standard that uses OS deep links for discovery, X25519 ECDH for an encrypted channel, and JSON-RPC for the four core methods (`authorize`, `reauthorize`, `sign_transactions`, `sign_and_send_transactions`).

An MWA wallet built in Flutter splits roughly 80 percent Dart, 20 percent native configuration. The hardest part is not the protocol — it is the approval UI. Showing the user what they are about to sign, in a way that defends against both decoded-but-malicious transactions and undecoded unknown ones, is the wallet industry's open problem. Simulation-first design (display the net balance change, decode what you can, flag what you cannot) is the current best practice.

Solfare does not implement MWA. Doing so would roughly double the project's scope and require sustained Swift, Kotlin, and Dart work plus a real test harness. But you now know what would be involved, what it would cost, and what to defend against. If you ship a Solana wallet that anyone else uses, you will eventually ship MWA. This chapter is the map.